

Bono 2 - Combinaciones

Diseñe un programa que reciba n y r , con $0 \leq r \leq n$, y calcule el número de formas de escoger r objetos entre n objetos distintos sin importar el orden:

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}$$

El programa debe permitir:

1. Calcular $\binom{n}{r}$ para cualquier entrada válida
2. Verificar automáticamente la identidad $\binom{n}{r} = \binom{n}{n-r}$
3. Generar la fila n del triángulo de Pascal
4. Imprimir ejemplos de uso

Extensión opcional: generar todo el triángulo de Pascal hasta la fila n .

Descripción matemática

Combinaciones

Las combinaciones son una técnica de conteo para averiguar todas las formas en las que se puede seleccionar k elementos de un conjunto de n elementos. La fórmula para lograr esto es:

$$nCk = \frac{n!}{k!(n-k)!}$$

Nótese que es igual a la fórmula de la k -permutación, pero con un $k!$ en el denominador. Esto equivale a tomar todas las combinaciones y dividirlo por las $k!$ posibles órdenes de la selección, para contar solo las selecciones únicas. En otras palabras:

$$nCk = \frac{P(n, k)}{k!}$$

Es necesario que $k \leq n$, porque si no, $n - k$ es negativo, y por ende $(n - k)!$ no existe.

En la combinación el orden de selección de los elementos nunca importa. La permutación puede considerarse un superset de las combinaciones donde el orden sí importa.

Otra notación para las combinaciones es la llamada **notación binomial**:

$$nCk = \binom{n}{k}$$

Esta notación también muy común está asociada con el **teorema de Newton** o teorema del binomio, que usa estos coeficientes para determinar los coeficientes de la potencia de un binomio. No por nada se llaman **coeficientes binomiales**.

Triángulo de Pascal

El triángulo de Pascal es una estructura matemática para ordenar los valores de los coeficientes de un triángulo de números. Cada número es la suma de los dos números directamente encima de él. La fila n del triángulo de Pascal corresponde a los coeficientes binomiales $\binom{n}{k}$ para $k = 0, 1, \dots, n$.

				1				
			1		1			
		1		2		1		
	1		3		3		1	
	1	4		6		4		1
	1	5	10		10	5		1
	1	6	15	20		15	6	1
	1	7	21	35	35	21	7	1

Figura 1: Triángulo de Pascal hasta el nivel 8

								$\binom{0}{0}$
						$\binom{1}{0}$	$\binom{1}{1}$	
				$\binom{2}{0}$	$\binom{2}{1}$	$\binom{2}{2}$		
		$\binom{3}{0}$	$\binom{3}{1}$	$\binom{3}{2}$	$\binom{3}{3}$			
	$\binom{4}{0}$	$\binom{4}{1}$	$\binom{4}{2}$	$\binom{4}{3}$	$\binom{4}{4}$			
$\binom{5}{0}$	$\binom{5}{1}$	$\binom{5}{2}$	$\binom{5}{3}$	$\binom{5}{4}$	$\binom{5}{5}$			
$\binom{6}{0}$	$\binom{6}{1}$	$\binom{6}{2}$	$\binom{6}{3}$	$\binom{6}{4}$	$\binom{6}{5}$	$\binom{6}{6}$		
$\binom{7}{0}$	$\binom{7}{1}$	$\binom{7}{2}$	$\binom{7}{3}$	$\binom{7}{4}$	$\binom{7}{5}$	$\binom{7}{6}$	$\binom{7}{7}$	

Figura 2: Triángulo de Pascal, en formato de coeficiente binomial

De aquí podemos derivar algunas propiedades:

1. Como el triángulo es simétrico, los lados a la misma distancia de los extremos son iguales en valor. Esto es la **identidad de Pascal**:

$$\binom{n}{k} = \binom{n}{n-k}$$

2. Los extremos de las filas son 1, entonces:

$$\binom{n}{0} = \binom{n}{n} = 1$$

3. Sumar dos elementos consecutivos en una fila obtiene el elemento de debajo. Esta es la **regla de Pascal**:

$$\binom{n}{k} + \binom{n}{k+1} = \binom{n+1}{k+1}$$

Implementación

Combinaciones recursivas

De forma sencilla, sabemos que tenemos a la mano la librería `math`, la cual ofrece el método `math.comb`:

```
import math as m

print(m.comb(5, 3)) # -> 10
```

Antes de proceder con el código de nuestra versión, por supuesto, vamos a averiguar como funcionan las combinaciones para desarrollar optimizaciones.

Usando la regla de Pascal sabemos que:

$$\binom{n}{k} + \binom{n}{k-1} = \binom{n+1}{k+1}$$

Usando esto podemos derivar una forma recursiva de la combinación. La puedes verificar visualmente usando el triángulo de Pascal:

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

Por supuesto, si es recursivo, necesitamos un caso base. Los casos base son:

$$\binom{n}{0} = 1, \quad \binom{n}{n} = 1$$

Implementado en Python nos queda así. Usamos anotaciones de tipo para mejor legibilidad:

```
def recursive_combination(n: int, k: int) -> int:
    """Calcula combinaciones de forma recursiva usando la regla de Pascal"""

    # Error de tipo si no son enteros
    if type(n) is not int or type(k) is not int:
        raise TypeError(f"Valores no válidos: {n}, {k}")

    # Error si alguno es menor que cero o si n es menor que k
    if n < 0 or k < 0 or n < k:
        raise ValueError(f"Valores no válidos: {n}, {k}")

    # Caso base
    if k == 0 or k == n:
        return 1

    # Recursión
    return recursive_combination(n - 1, k) + recursive_combination(n - 1, k - 1)
```

Sin embargo nos pueden preocupar los casos grandes, porque esta función es recursiva y se presta para altos consumos de memoria, calculos duplicados y un riesgo de `RecursionError` en Python.

Usando memoización

Para solucionar los problemas de la recursión, podemos usar la **memoización** para guardar los resultados de las combinaciones ya calculadas y evitar cálculos repetidos:

```
cached = {}
def memoized_combination(n: int, k: int) -> int:
    if (n, k) in cached:
        return cached[(n, k)]

    # Error de tipo si no son enteros
    if type(n) is not int or type(k) is not int:
        raise TypeError(f"Valores no válidos: {n}, {k}")

    # Error si alguno es menor que cero o si n es menor que k
    if n < 0 or k < 0 or n < k:
        raise ValueError(f"Valores no válidos: {n}, {k}")

    # Caso base
    if k == 0 or k == n:
        cached[(n, k)] = 1
        return 1

    # Recursión
    result = memoized_combination(n - 1, k) + memoized_combination(n - 1, k - 1)
    cached[(n, k)] = result
    return result
```

Esto reducirá los cálculos enormemente, pero igual el consumo de memoria empeora. Sin embargo a la hora de reutilizar ahorra muchísimo tiempo, especialmente para casos grandes donde se generan lotes de resultados precalculados y listos para la próxima.

Combinaciones iterativas

Para una última versión de la combinatoria, volvamos a la fórmula original:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Desenrollando el factorial $n!$ hasta llegar a $(n-k)!$ desarrollamos:

$$\begin{aligned}\binom{n}{k} &= \frac{n!}{k!(n-k)!} \\ \binom{n}{k} &= \frac{n(n-1)(n-2) \cdot \dots \cdot (n-k+1)\cancel{(n-k)!}}{k!\cancel{(n-k)!}} \\ \binom{n}{k} &= \frac{n(n-1)(n-2) \cdot \dots \cdot (n-k+1)}{k!}\end{aligned}$$

Y ahora podemos emparejar cada término del factorial $k!$ con cada término del numerador para obtener dos secuencias:

$$\binom{n}{k} = \frac{n}{k} \cdot \frac{n-1}{k-1} \cdot \frac{n-2}{k-2} \cdot \dots \cdot \frac{n-k+1}{1}$$

$$\binom{n}{k} = \frac{n}{1} \cdot \frac{n-1}{2} \cdot \frac{n-2}{3} \cdot \dots \cdot \frac{n-k+1}{k}$$

Podemos generalizarlo con un producto para cada secuencia respectivamente:

$$\binom{n}{k} = \prod_{i=0}^{k-1} \frac{n-i}{k-i} = \prod_{i=1}^k \frac{n-i+1}{i}$$

Esta es la forma más eficiente de calcular la combinación, al reducirlo a un producto. Sin riesgo de RecursionError y con un consumo de memoria constante. Usaremos la segunda versión porque es más adecuada para cálculo basado en enteros:

```
def iterative_combination(n: int, k: int) -> int:
    """Calcula combinaciones de forma iterativa usando la fórmula desenrollada"""

    # Error de tipo si no son enteros
    if type(n) is not int or type(k) is not int:
        raise TypeError(f"Valores no válidos: {n}, {k}")

    # Error si alguno es menor que cero o si n es menor que k
    if n < 0 or k < 0 or n < k:
        raise ValueError(f"Valores no válidos: {n}, {k}")

    # Optimizamos usando la identidad de Pascal
    k = min(k, n - k)

    result: int = 1
    for i in range(1, k + 1):
        result = result * (n - i + 1) // i

    return result
```

Nótese que incluimos una optimización:

```
# Optimizamos usando la identidad de Pascal
k = min(k, n - k)
```

Como lo dice el comentario, es una aplicación de la **identidad de Pascal**:

$$\binom{n}{k} = \binom{n}{n-k}$$

Esto nos permite reducir el número de términos a calcular, usando el número más pequeño de los dos disponibles sin alterar el resultado. Si funciona, esta implementación es la prueba.

Triángulo de Pascal

Mientras me encontraba escribiendo este artículo para el bono, encontré en internet esta función en Typst para generar los triángulos de Pascal de más arriba:

[Fuente](#)

```
#let pascal_triangle(n) = {
  set align(center)
  let row = ()
  for r in range(0, n) {
    // step the row
    for i in range(row.len() - 1, 0, step: -1) {
      row.at(i) = row.at(i) + row.at(i - 1)
    }
    row.push(1)
    // print the row
    grid(
      columns: row.len() * (32pt,),
      align: center,
      //stroke : 0.2pt,
      ..row.map(str)
    )
  }
}
```

Por lo que decidiré reutilizarla en Python para hacer el triángulo en consola. Su funcionamiento se basa en la propiedad:

$$\binom{n+1}{k+1} = \binom{n}{k} + \binom{n}{k+1}$$

Con la cual sobrescribe una fila para armar la siguiente, elemento por elemento, luego le añade un uno al final para completarla, imprime y repite el proceso. De esta forma, cada fila se construye a partir de la anterior, y se imprime a medida que se va construyendo:

```
def pascal_triangle(n: int):
    """Imprime el triángulo de Pascal con n filas"""

    row = []
    for _ in range(n):
        # Crea la fila
        for i in range(len(row) - 1, 0, -1):
            row[i] = row[i] + row[i - 1]
        row.append(1)

        # Imprime la fila
        print("\t".join(str(x) for x in row))
```

Aplicación de consola

Para una aplicación de consola solo usaremos `prints` e `inputs` para seguir un flujo de usuario básico. También se incluyen verificaciones de errores bien manejados y un código limpio y modular.

El programa estará en el archivo `bono_2_combinaciones/main.py`.

Pruebas y casos especiales

Para probar este sistema vamos a probar cuatro casos, con varios ejemplos diferentes para cada caso y por cada algoritmo:

1. Números correctos en un rango común (1-100)
2. Uso de negativos y del cero
3. Números grandes (superiores a 1000)
4. Entradas incorrectas (rango o tipo incorrecto)

Este programa estará en el archivo `bono_2_combinaciones/tests.py`

Comentarios y extras

1. Lee los comentarios del primer bono.
2. Al igual que con las permutaciones, la versión iterativa es probablemente la más adecuada por consumo de memoria, rendimiento y menos desperdicio.
3. Tenía pensado hablar de optimización de llamada de cola acá también pero preferí abordar otras formas de optimización y así no tener tantas variantes a comparar.
4. Me he encontrado con buenos recursos para la escritura de estos dos bonos, como [esta gramática de lenguaje para Sublime](#) que me permitió añadir resaltado de código estilo VSCode. También disponible en [este gist con la versión en tmTheme](#) (el formato que Typst usa para sus temas de sintaxis), también disponible en este repo (`vscode_darkplus.tmTheme`).
5. Cada día me arrepiento menos de no haber usado libretas de Jupyter para estos bonos. Hasta dan ganas de crear una alternativa a Jupyter que sí sirva bien. Quizá aprenda Rust para lograrlo. O simplemente use **Weave.jl**.

Referencias

- Khan Academy. (s.f.). *Introducción a las combinaciones* [Video]. <https://es.khanacademy.org/math/precalculus/x9e81a4f98389efdf:prob-comb/x9e81a4f98389efdf:combinations/v/introduction-to-combinations>
- Khan Academy. (s.f.). *Fórmula de las combinaciones* [Video]. <https://es.khanacademy.org/math/precalculus/x9e81a4f98389efdf:prob-comb/x9e81a4f98389efdf:combinations/v/combinacion-formula>
- Wikipedia. (2024). *Triángulo de Pascal*. https://es.wikipedia.org/wiki/Triángulo_de_Pascal
- Wikipedia. (2024). *Coeficiente binomial*. https://es.wikipedia.org/wiki/Coeficiente_binomial
- Universidad Industrial de Santander. (s.f.). *Análisis combinatorio*. Repositorio Noesis. <https://noesis.uis.edu.co/server/api/core/bitstreams/d0c3f853-dc1b-4742-9038-6e6b91650b95/content>